WORKSHEET **P8** Pertemuan 8 · JavaScript Modern ES6+, Struktur Data, dan Array Methods

Membuat Halaman Profil yang Datanya Bergerak

Pengembangan Aplikasi Berbasis Web · SIF302 · Semester Gasal 2026/2027

NAMA		NIM	
KELAS		TANGGAL	

Dikerjakan sendiri, bukan berpasangan. Setiap mahasiswa mengerjakan satu halaman profil miliknya sendiri. Anda bebas memilih isi halaman, tetapi harus bisa menjelaskan setiap baris kode yang Anda tulis.

Capaian pembelajaran	Yang dinilai hari ini
Mahasiswa mampu menuliskan data halaman sebagai nilai JavaScript — variabel, fungsi, objek, dan array — lalu mengolahnya dengan array methods sehingga isi halaman tidak lagi ditulis tangan di dalam HTML.	1. JavaScript terhubung ke halaman dan berjalan tanpa galat di Console. 2. Data halaman (identitas, keahlian, daftar proyek) tersimpan sebagai variabel, objek, dan array — bukan sebagai teks di dalam HTML. 3. Minimal dua fungsi murni bekerja dan hasilnya bisa dilihat. 4. Array methods map, filter, dan find dipakai pada data yang benar. 5. Galat dibaca dari Console dan dijelaskan, bukan dihapus dengan coba-coba.

Aturan pengerjaan

Hal	Ketetapan
Bentuk kerja	Individu. Satu mahasiswa, satu halaman, satu topik.
Perkakas	VS Code + peramban Chrome atau Firefox. Paket ekstensi Live Server dipasang di VS Code (cara lain ada di lembar A).
Berkas awal	Hasil pekerjaan Anda sendiri dari Pertemuan 6 — profil.html beserta seluruh berkas CSS-nya — disalin ke folder worksheet-p8/. Tidak ada berkas contoh, tidak ada kerangka yang disediakan.
Waktu	90 menit di kelas. Bila belum selesai, dilanjutkan di luar kelas paling lambat pukul 23.59 pada hari yang sama.
Penyerahan	Seluruh isi worksheet-p8/ diunggah ke repositori GitHub Anda sendiri, di dalam folder semester ~/pabw/. Periksa halaman repositori Anda, bukan hanya folder di laptop.

Anggaran waktu 90 menit

Menit	Lembar	Yang Anda kerjakan	Kalau waktu habis, berhenti di sini
0–18	A	Salin hasil Pertemuan 6 ke worksheet-p8/, buat folder js/, hubungkan app.js, jalankan halaman lewat server lokal	Berkas terhubung dan Console bersih
18–38	B	const dan let, template literal, nilai bawaan ?? dan akses aman ?.	Identitas profil tersusun dari variabel, bukan teks tertulis
38–56	C	Dua fungsi murni: pembuat kalimat perkenalan dan pemformat daftar keahlian	Dua fungsi bekerja dan hasilnya terlihat di Console
56–80	D	Array of object, map, filter, dan find	console.table menampilkan seluruh data; filter menghasilkan subset yang benar
80–100	E	Membaca galat: console, breakpoint, dan tiga kasus sulit	Tiga kasus ditangani + tangkapan layar Console
100–150	F	Periksa satu per satu, satu baris untuk diingat, tiket keluar, penilaian mandiri, dan bagian README	Lembar F terisi dan tersimpan

Selalu kerjakan berurutan. Setiap lembar mengandaikan lembar sebelumnya sudah selesai. Kalau Anda melompat, galat yang muncul akan sulit dilacak karena sebabnya ada di lembar yang belum Anda kerjakan. Kolom paling kanan bukan batas waktu — hanya penunjuk laju.

SATU BARIS PENGHUBUNG, SELURUH BERKAS JADI HIDUP

Sampai Pertemuan 6 halaman Anda sudah menyesuaikan diri pada setiap lebar layar, tetapi isinya masih tertulis tetap di dalam HTML. Mengubah satu nama berarti menyunting HTML. Mulai hari ini, isi halaman itu dipindahkan ke JavaScript sebagai data.

A.1 Prasyarat: halaman harus dibuka lewat server lokal

Berkas yang punya <script type="module"> tidak boleh dibuka dengan klik dua kali — peramban akan menolaknya karena alamatnya berawalan file://. Buka lewat salah satu cara berikut, dan pastikan yang terjadi: alamat di bilah peramban berawalan http://.

Cara	Yang dilakukan	Alamat yang muncul di bilah peramban
Live Server (VS Code)	Pasang ekstensi Live Server, buka folder worksheet-p8/ di VS Code, klik kanan profil.html → Open with Live Server	
Python	Dari terminal, masuk ke folder worksheet-p8/ lalu jalankan python3 -m http.server 8000	

Halaman akan memuat ulang sendiri setiap kali Anda menyimpan berkas. Itu bukan kerusakan — itu memang perilaku Live Server. Tanpa itu, Anda harus menekan F5 sendiri setiap kali selesai menulis satu baris.

A.2 Dua baris yang menyambungkan halaman dengan skrip

Tulis tepat satu baris <script> sebelum </body>. Baris itu memuat tag pembuka dan tag penutup sekaligus; jangan menuliskannya dua kali. Atribut type="module" wajib ada, dan urutannya tidak boleh ditukar.

TULIS DI DALAM PROFIL.HTML, TEPAT SEBELUM TUTUP BODY

```
  <!-- ... isi halaman Anda yang sudah ada ... -->

  <script type="module" src="js/app.js"></script>
</body>
```

Bagian baris	Untuk apa	Akibat bila salah
type="module"	Menyalakan aturan modul: nama variabel tidak bocor ke jendela peramban, dan import/export bisa dipakai (Pertemuan 11)	Tanpa itu skrip tetap jalan hari ini, tetapi akan gagal di Pertemuan 11
src="js/app.js"	Menunjuk berkas skrip yang Anda tulis, relatif terhadap profil.html	Jalur salah → Console menampilkan 404, halaman tampak normal tetapi skrip tidak jalan
Letak sebelum </body>	Elemen halaman sudah ada saat skrip membaca DOM (Pertemuan 9)	Di dalam <head>, skrip berjalan sebelum elemen ada dan membaca nilai kosong

A.3 Susunan folder setelah lembar ini

Buat folder js/ di dalam worksheet-p8/, lalu tambahkan satu berkas kosong bernama app.js. Berkas hasil salinan Pertemuan 6 tidak diubah sama sekali — halaman itu tetap tampil seperti sebelumnya sampai Anda menambahkan data di lembar B.

Sudah dikerjakan	Tanda
Folder worksheet-p8/ berisi profil.html dan seluruh berkas CSS dari Pertemuan 6	
Folder js/ sudah dibuat dan berisi app.js	
Baris <script type="module" src="js/app.js"></script> sudah ada sebelum </body>	
Halaman dibuka lewat server lokal dan alamat di bilah peramban berawalan http://	
Console tidak menampilkan pesan merah	

Kalau halaman tampak biasa saja padahal app.js kosong, itu tanda yang baik. Artinya skrip tidak mengubah apa pun yang belum Anda tulis.

SIMPAN KE GIT — SETELAH LEMBAR A

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P8 — hubungkan skrip ke halaman
```

Lembar A sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

A — simpan perubahan

Cara memastikan: buka Console. Tidak ada tulisan merah, dan tidak ada tulisan 404 di samping nama berkas app.js.

Lembar B — Data halaman sebagai variabel

NILAI YANG DIKETIK TANGAN AKAN LAPUK, NILAI DARI VARIABEL TIDAK

Sekarang pindahkan identitas halaman Anda dari HTML ke JavaScript. Yang dipindah hanya isinya; struktur HTML-nya tetap. Bila identitas itu muncul di dua tempat — misalnya di <title> dan di dalam <header> — cukup tulis sekali sebagai variabel, lalu pakai di kedua tempat. Di situlah gunanya.

B.1 const dan let: deklarasi yang menyatakan niat

JavaScript punya dua cara menampung nilai, dan pilihannya bukan selera. Pakai const bila nilai itu tidak akan ditunjuk ulang; pakai let hanya bila Anda memang akan mengubahnya.

Deklarasi	Pakai untuk	Yang terjadi bila dilanggar
const	Nilai yang tidak ditunjuk ulang: data profil, daftar keahlian, daftar proyek, referensi elemen	Console menampilkan TypeError: Assignment to constant variable
let	Penghitung, penanda pilihan yang berubah, hasil sementara di dalam fungsi	Tidak ada — let memang untuk diubah
var	Tidak dipakai lagi	Bocor keluar dari blok dan menyulitkan pelacakan; tidak dipakai di pertemuan ini

NILAI, TIPE, DAN TYPEOF

```
const nama = "Ayu";          // teks
const jumlahProyek = 3;      // angka, bukan "3"
let pilihanAktif = "semua";  // akan berubah saat disaring

console.log(typeof nama);         // "string"
console.log(typeof jumlahProyek); // "number"
console.log(typeof belumDibuat);  // undefined
```

B.2 Template literal: menyusun kalimat dari nilai

Menyambung teks dengan tanda + cepat menjadi sulit dibaca begitu nilainya lebih dari dua. Template literal memakai tanda petik miring (backtick) dan menaruh nilai di dalam ${ }. Bedanya penting: tanpa template literal, angka dan teks tersambung apa adanya.

Yang ditulis	Hasilnya	Catatan
'Nama: ' + nama	Nama: Ayu	Bekerja, tetapi setiap spasi harus Anda hitung sendiri
`Nama: ${nama}`	Nama: Ayu	Nilai apa pun otomatis menjadi teks
'1' + 1	11	Tanda + dengan teks berarti menyambung, bukan menjumlahkan
`${1 + 1}`	2	Di dalam ${ } isinya tetap dihitung sebagai angka

TUJUAN LEMBAR B

```
const profil = {
  nama: "Nama Anda",
  peran: "Mahasiswa Informatika",
  keahlian: ["HTML", "CSS", "JavaScript"],
};

const kalimat = `Nama saya ${profil.nama}, dan saya belajar ${profil.keahlian.length} hal.`;
console.log(kalimat);
```

B.3 Enam kasus JavaScript yang paling sering menyesatkan

Keenam baris ini yang paling sering muncul di Console mahasiswa pada pertemuan ini. Bacalah sebelum menulis kode, bukan sesudah galat muncul.

Yang sering ditulis	Kenapa menyesatkan	Yang benar
if (nama == "")	== menyamakan nilai setelah mengubah tipe — 0, "", dan null bisa dianggap sama	Pakai === dan !== supaya tipe ikut dibandingkan
isi = input.value || "-"	|| menganggap 0 dan "" sebagai kosong, padahal keduanya nilai sah	Pakai ?? bila Anda hanya ingin menangkap null dan undefined
profil.alamat.kota	Bila alamat belum ada, baris ini melempar TypeError dan menghentikan seluruh skrip	Pakai profil.alamat?.kota — hasilnya undefined, bukan galat
if (0) dan if ("")	Nol dan teks kosong dinilai salah (falsy), padahal itu nilai yang sah	Bandingkan langsung: if (jumlah === 0)
var di dalam { }	var tidak terkurung blok, jadi bocor keluar dan bisa menimpa variabel lain	Pakai const atau let
input.value + 1	Nilai dari kolom isian selalu teks — hasilnya "101", bukan 101	Ubah dulu: Number(input.value) atau Number.parseInt(input.value, 10)

B.4 Tulis data profil saya

Data	Nama variabel yang saya pakai	Contoh isi
Nama lengkap		
Kalimat peran		Mahasiswa Informatika yang belajar front-end
Daftar keahlian (minimal tiga)		
Satu nilai angka yang dipakai nanti		jumlah proyek yang sudah dibuat

Isi halaman Anda sendiri, bukan contoh di atas. Topik yang berbeda antar mahasiswa membuat pekerjaan tidak bisa saling disalin, dan itu memang tujuannya.

SIMPAN KE GIT — SETELAH LEMBAR B

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P8 — data profil jadi variabel
```

Lembar B sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

B — simpan perubahan

Cara memastikan: buka Console dan panggil salah satu variabel Anda. Nilainya harus muncul, bukan undefined.

Lembar C — Dua fungsi murni

KODE DIKATAKAN SEKALI, DIPAKAI BERKALI-KALI

Variabel menyimpan nilai. Fungsi menyimpan pekerjaan. Pada lembar ini Anda menulis dua fungsi yang benar-benar Anda pakai: satu untuk menyusun kalimat perkenalan, satu untuk merapikan daftar keahlian menjadi satu baris teks.

C.1 Tiga bentuk fungsi, satu kegunaan

Ketiganya sama-sama membuat fungsi; yang berbeda adalah cara menulisnya. Pilih satu bentuk dan pakai konsisten di seluruh berkas Anda.

Bentuk	Tulisan	Kapan pantas dipakai
Deklarasi	function buatPerkenalan(nama) { }	Paling mudah dibaca; boleh dipanggil sebelum barisnya ditulis
Expression	const buatPerkenalan = function (nama) { };	Bila Anda ingin fungsinya menjadi nilai biasa
Arrow	const buatPerkenalan = (nama) => { };	Paling ringkas; dipakai hampir selalu sebagai argumen map dan filter

DUA FUNGSI YANG HARUS ANDA TULIS

```
// 1. Menyusun kalimat perkenalan dari satu object
function buatPerkenalan({ nama, peran }) {
  return `${nama} — ${peran}`;
}

// 2. Merapikan daftar keahlian menjadi satu baris teks
const formatKeahlian = (daftar) => daftar.join(" · ");

console.log(buatPerkenalan(profil));
console.log(formatKeahlian(profil.keahlian));
```

C.2 Parameter, nilai bawaan, dan argumen objek

Parameter adalah nama yang Anda pakai di dalam fungsi; argumen adalah nilai yang Anda kirim saat memanggilnya. Bila fungsi memerlukan lebih dari dua argumen, kirim satu objek saja — urutannya tidak lagi mudah tertukar.

Cara menulis	Contoh pemanggilan	Keunggulannya
Parameter tunggal	buatPerkenalan("Ayu")	Paling sederhana
Nilai bawaan	buatPerkenalan("Ayu", "Informatika")	Argumen kedua boleh dilewati tanpa menjadi undefined
Satu objek	buatPerkenalan({ nama: "Ayu", peran: "mahasiswa" })	Nama argumen terlihat di tempat pemanggilan, urutan tidak bisa tertukar

C.3 Fungsi murni: hasilnya bisa ditebak

Fungsi murni hanya bergantung pada argumennya dan tidak mengubah apa pun di luar dirinya. Praktisnya: fungsi yang sama, dengan argumen yang sama, selalu memberi hasil yang sama. Ini yang membuat kode Anda bisa diperiksa — oleh Anda sendiri, dua pekan kemudian.

Pertanyaan	Fungsi murni	Bukan fungsi murni
Hasilnya bergantung pada apa?	Hanya pada argumen	Juga pada variabel di luar atau isi halaman

Pertanyaan	Fungsi murni	Bukan fungsi murni
Mengubah sesuatu di luar dirinya?	Tidak	Ya — menulis ke Console pun dihitung mengubah dunia luar
Aman dipanggil dua kali?	Ya, hasilnya sama	Belum tentu — pemanggilan kedua bisa memberi hasil berbeda

C.4 Enam kasus fungsi yang sering muncul

Yang sering ditulis	Kenapa menyesatkan	Yang benar
Fungsi tanpa return	Hasilnya selalu undefined, walau di dalamnya ada perhitungan	Tulis return di baris terakhir yang menghasilkan nilai
return di dalam forEach	forEach tidak meneruskan nilai keluar; return-nya hanya menghentikan satu putaran	Pakai for...of bila butuh berhenti, atau map bila butuh menghimpun hasil
map tanpa return	Isi array hasilnya undefined semua, panjangnya tetap benar sehingga sulit disadari	Pastikan callback map mengembalikan nilai
Nama fungsi tanpa kata kerja	Peran fungsi tidak terbaca	Mulai dengan kata kerja: ambil, buat, hitung, format
Satu fungsi tiga pekerjaan	Sulit diperiksa dan sulit dipakai ulang	Pecah sampai satu fungsi satu pekerjaan
Variabel di luar fungsi	Fungsi jadi tidak murni dan sulit diuji	Kirim nilainya sebagai parameter

Uji cepat tanpa alat apa pun. Panggil fungsi Anda tiga kali dengan argumen berbeda di Console. Kalau hasilnya masuk akal untuk ketiganya, fungsi itu cukup baik untuk dilanjutkan.

SIMPAN KE GIT — SETELAH LEMBAR C

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P8 — dua fungsi murni
```

Lembar C sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

C — simpan perubahan

Cara memastikan: panggil kedua fungsi Anda di Console dengan argumen berbeda. Hasilnya muncul dan sesuai dugaan Anda.

Lembar D — Struktur data dan array methods

DATA DULU, TAMPILAN BELAKANGAN

Satu nilai sudah bisa ditampung variabel. Sekarang tempatkan seluruh isi halaman sebagai data: satu objek untuk identitas, satu array of object untuk daftar keahlian dan daftar proyek. Setelah ini Anda tidak lagi menyunting HTML untuk mengubah isi.

D.1 Object dan array of object

Object menyimpan nilai berlabel; array menyimpan urutan. Daftar proyek Anda adalah array yang setiap isinya sebuah object — dan itulah bentuk yang paling sering Anda temui di pekerjaan nyata.

Yang ditulis	Artinya	Cara membaca nilainya
profil.nama	Nilai berlabel nama di dalam objek profil	Titik dipakai bila labelnya Anda ketahui sendiri
daftarProyek[0]	Object pertama di dalam array daftarProyek	Kurung siku dipakai untuk urutan (indeks mulai dari 0)
daftarProyek[0].judul	Nilai judul dari object pertama	Keduanya digabung
profil["nama"]	Sama dengan profil.nama	Kurung siku berguna bila labelnya baru diketahui saat program berjalan

STRUKTUR DATA YANG DITUJU

```
const profil = {
  nama: "Nama Anda",
  peran: "Mahasiswa Informatika",
  keahlian: ["HTML", "CSS", "JavaScript"],
};

const daftarProyek = [
  { judul: "Halaman Profil", tahun: 2026, selesai: true },
  { judul: "Katalog Produk", tahun: 2026, selesai: false },
];
```

D.2 Empat kesalahan yang menghabiskan waktu paling banyak

Salah	Benar	Kenapa
const salinan = profil	const salinan = { ...profil }	Yang disalin hanya penunjuknya. Mengubah salinan ikut mengubah aslinya.
daftarProyek.sort()	const urut = [...daftarProyek].sort()	sort mengubah array aslinya, bukan menghasilkan array baru.
daftarProyek.map(p => p.judul === 'A')	daftarProyek.filter(p => p.judul === 'A')	map mempertahankan jumlah isi; untuk menyaring pakai filter.
daftarProyek['0']	daftarProyek[0]	Array memakai indeks berupa angka, bukan teks.

D.3 map, filter, dan find: kapan memakai yang mana

Ketiganya menerima satu fungsi sebagai argumen dan memanggilnya untuk setiap isi array. Yang membedakan adalah apa yang mereka kembalikan.

Method	Yang dikembalikan	Dipakai untuk	Contoh hasil
map	Array baru, panjangnya sama	Mengubah setiap isi	[1, 2, 3] → [2, 4, 6]
filter	Array baru, panjangnya bisa lebih pendek	Menyaring isi	[1, 2, 3] → [2, 3]

Method	Yang dikembalikan	Dipakai untuk	Contoh hasil
find	Satu isi, atau undefined	Mengambil isi pertama yang cocok	[1, 2, 3] → 2
reduce	Satu nilai	Menghimpun seluruh isi menjadi satu angka atau teks	[1, 2, 3] → 6

YANG HARUS TERCETAK DI CONSOLE

```
console.table(profil.keahlian);
console.table(daftarProyek);

const selesai = daftarProyek.filter((proyek) => proyek.selesai);
console.table(selesai);

const katalog = daftarProyek.find((proyek) => proyek.judul === "Katalog Produk");
console.log(katalog);
```

D.4 Isi tabel pemeriksaan data saya

Isi apa adanya. Tabel yang jujur lebih berguna daripada tabel yang semuanya benar.

Yang diperiksa	Hasil yang benar	Hasil saya
console . table	Seluruh isi tampil sebagai tabel, jumlah barisnya sama dengan isi array	
filter pada satu label	Hanya isi yang cocok yang tersisa, dan jumlahnya masuk akal	
find satu isi	Yang muncul satu object; kalau tidak ada, hasilnya undefined	
map pada daftarProyek	Panjang array hasil sama dengan array asal	
Data asli setelah sort	Urutan daftarProyek tidak berubah karena Anda memakai salinan	

SIMPAN KE GIT — SETELAH LEMBAR D

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P8 — data halaman jadi array of object
```

Lembar D sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

D — simpan perubahan

Cara memastikan: jalankan console.table pada data Anda, lalu bandingkan jumlah barisnya dengan jumlah isi array di berkas app.js.

Lembar E — Membaca galat, bukan menebaknya

GALAT YANG TERBACA ADALAH GALAT YANG CEPAT SELESAI

Mulai pertemuan ini kode Anda bisa gagal, dan itu bagian dari pekerjaan. Yang dinilai bukan keberhasilan menghindari galat, melainkan cara Anda membacanya. Jangan menghapus baris sampai galatnya hilang tanpa tahu sebabnya.

E.1 Membaca pesan galat dari baris pertama

Pesan galat selalu menyebut tiga hal: apa yang salah, di berkas mana, dan baris berapa. Bacalah dari baris pertama — sisa pesannya hanya menjelaskan perjalanan menuju baris itu.

Yang dipakai	Untuk apa	Kapan dipakai
console.log(nilai)	Melihat satu nilai apa adanya	Memeriksa dugaan: "apakah variabel ini benar-benar berisi?"
console.table(array)	Melihat seluruh isi array atau object sebagai tabel	Memeriksa data, bukan mencari kesalahan
console.error(pesan)	Menandai pesan Anda sendiri sebagai galat	Menandai jalan buntu atau cabang yang tidak seharusnya tercapai
Console → pesan merah	Menunjuk berkas dan baris tempat galat muncul	Selalu langkah pertama setelah halaman tidak berperilaku seperti dugaan

E.2 Menjalankan halaman lewat server lokal, bukan file://

Bila Console menampilkan pesan yang menyebut CORS, module, atau origin, penyebabnya hampir selalu cara Anda membuka halaman — bukan kode Anda. Tutup tab itu, lalu buka kembali lewat Live Server atau python3 -m http.server seperti pada lembar A.

Gejala di Console	Penyebabnya	Yang dilakukan
Access to script ... has been blocked by CORS policy	Halaman dibuka lewat file://, sehingga peramban menolak memuat modul	Buka lewat alamat http:// seperti pada A.1
Failed to load resource: 404 (Not Found)	Jalur di atribut src tidak cocok dengan letak berkasnya	Periksa huruf besar-kecil nama berkas dan letak folder js/
Unexpected token '<'	Berkas yang dimuat bukan JavaScript	Pastikan src menunjuk app.js, bukan profil.html

E.3 Debugging dengan breakpoint

Console cocok untuk memeriksa satu nilai. Untuk melihat nilai yang berubah di tengah putaran, hentikan programnya. Klik nomor baris di panel Sources — titik merah itulah breakpoint — lalu muat ulang halaman.

Yang dilakukan	Di mana	Yang Anda lihat
Klik nomor baris	DevTools → Sources → berkas app.js	Titik merah; program berhenti di baris itu
Muat ulang halaman	Peramban	Eksekusi berhenti dan baris itu disorot
Langkah per baris	Tombol Step over (F10)	Jalannya program, satu baris demi satu baris
Periksa nilai	Panel Scope, atau tulis nama variabel di Console	Nilai variabel tepat pada saat itu — bukan nilai akhirnya

E.4 Tiga kasus sulit dan cara menanganinya

Yang muncul	Artinya	Yang dilakukan

Yang muncul	Artinya	Yang dilakukan
undefined pada nilai yang seharusnya ada	Labelnya salah tulis, atau datanya memang belum diisi	Bandingkan nama label dengan yang ada di berkas app.js huruf per huruf
Cannot read properties of null	querySelector tidak menemukan elemennya, jadi hasilnya null	Periksa penulisannya terhadap id atau class di HTML, dan pastikan skrip dijalankan setelah elemen itu ada
Nilai dari kolom isian tidak bisa dihitung	Nilai dari input selalu teks, bukan angka	Ubah dulu dengan Number(input.value); periksa hasilnya dengan console.log

E.5 Catat galat yang saya temui

Pesan galat (tulis apa adanya)	Baris ke berapa	Sebabnya	Yang saya ubah

Simpan tiga tangkapan layar Console: satu saat galat muncul, satu setelah diperbaiki, satu saat data tampil sebagai tabel. Ketiganya Anda perlukan di lembar F.

SIMPAN KE GIT — SETELAH LEMBAR E

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P8 — galat dibaca, bukan dihapus
```

Lembar E sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

E — simpan perubahan

Cara memastikan: Console Anda bersih, dan tabel di E.5 terisi dengan sebab yang benar — bukan dengan "sudah diperbaiki" saja.

TIKET KELUAR

Dikerjakan sendiri, tanpa berdiskusi. Lembar ini mengukur pemahaman Anda, bukan kecepatan Anda menyalin jawaban teman.

1. Tulis satu potong data di halaman Anda yang sekarang tersimpan sebagai variabel, dan sebutkan nama variabelnya. Apa yang harus Anda ubah kalau isinya berubah?

2. Apa bedanya const dan let? Berikan satu contoh pemakaian let di pekerjaan Anda.

3. Anda memakai { ...profil } untuk menyalin objek. Apa yang terjadi kalau tanda titik-tiga itu dihapus? Peragakan di Console, lalu tulis hasilnya.

4. Console menampilkan Cannot read properties of null pada baris yang memakai querySelector. Apa dua hal pertama yang Anda periksa, dan mengapa keduanya itu?

5. Mengapa nilai dari kolom isian tidak bisa langsung dijumlahkan? Tulis caranya supaya bisa.

F.1 Periksa satu per satu

Jangan menyerahkan pekerjaan sebelum menjalankan daftar ini. Semua yang diperiksa di bawah bisa Anda lihat sendiri dalam lima menit, dan semuanya benar-benar dinilai.

☐ Berkas profil.html dan folder js/ ada di repositori, terbuka tanpa galat di Console
☐ Skrip baris <script type="module" src="js/app.js"></script> ada sebelum </body>, tidak ditulis dua kali
☐ Data identitas, daftar keahlian, dan daftar proyek tersimpan sebagai const di app.js, tidak ditulis di HTML
☐ Fungsi minimal dua fungsi murni bekerja, masing-masing satu pekerjaan, memakai return
☐ Array methods map, filter, dan find dipakai pada data yang benar; data asli tidak berubah setelah salinan diurutkan
☐ Console tidak ada pesan merah; tiga tangkapan layar lembar E sudah disimpan
☐ Deklarasi AI bagian README menyebutkan bagian mana yang dibantu AI dan bagian mana yang

Anda kerjakan sendiri
☐ Git ada lebih dari satu commit hari ini, dan pesan commit-nya menjelaskan isinya

F.2 Satu baris untuk diingat

Tulis satu kalimat — bukan untuk dosen, untuk Anda sendiri dua pekan dari sekarang — tentang hal yang hari ini paling sering membuat Anda salah.

F.3 Penilaian mandiri

Isi perkiraan nilai Anda apa adanya sebelum menyerahkan lembar ini. Penilaian yang jujur membantu Anda mengetahui bagian mana yang belum kokoh. Kolom Bukti diisi nama berkas atau baris kode yang bisa dibuka dosen.

☐ Pengungkapan AI README menyebutkan bagian mana yang dibantu AI dan bagian mana yang saya kerjakan sendiri
☐ Keaslian topik dan isi halaman ini milik saya sendiri, tidak menyalin pekerjaan mahasiswa lain

Bagian	Bobot	Nilai saya	Bukti
Sintaks dan tipe data	15		
Fungsi murni	25		
Array methods	25		
Penanganan galat dan kebersihan Console	20		
Kebersihan kode, deklarasi AI, dan commit	15		
TOTAL	100		

F.4 Catatan untuk pengampu

Bagian tersulit hari ini:

Yang ingin saya bahas di kelas berikutnya:

Tanda tangan: _____ Tanggal: _____ Jam selesai: _____

SIMPAN KE GIT — SETELAH LEMBAR F

```
git add .
git commit -m "pesan Anda"
git push

# pesan commit yang disarankan:
# P8 — periksa, lalu simpan
```

Lembar F sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun — hanya memastikan pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

F — simpan perubahan

Cara memastikan: buka README di repositori Anda. Bagian Pertemuan 8 sudah memuat deklarasi AI, dan commit terakhir adalah commit terbaru.

Lampiran A — Rubrik penilaian

RUBRIK PENILAIAN total 100

Aspek	Bobot	Nilai 100 — sangat baik	Nilai 65 — memadai	Nilai 35 — kurang
Sintaks dan tipe data	15%	const dan let dipakai sesuai niat; template literal menggantikan penggabungan teks; perbandingan memakai ===; nilai angka benar-benar angka	Sebagian deklarasi masih memakai var, atau ada satu nilai angka yang tersimpan sebagai teks	Data belum dipindahkan dari HTML, atau skrip belum berjalan
Fungsi murni	25%	Dua fungsi dengan satu pekerjaan masing-masing, memakai return, tidak menyentuh nilai di luar dirinya, namanya memakai kata kerja	Fungsi bekerja tetapi salah satunya mengubah nilai di luar dirinya, atau namanya belum menggambarkan pekerjaannya	Fungsi tidak mengembalikan nilai, atau seluruh logika ditulis di satu tempat tanpa fungsi
Array methods	25%	map, filter, dan find dipakai pada data yang tepat; salinan memakai { ... }; data asli tidak berubah	Method dipakai tetapi ada yang tidak sesuai kegunaannya — misalnya map untuk menyaring	Array belum dipakai sebagai struktur data, atau method memakai pengulangan tangan
Penanganan galat dan kebersihan Console	20%	Console bersih; tiga kasus di E.4 ditangani dan sebabnya dijelaskan benar di E.5	Galat hilang tetapi sebabnya tidak dijelaskan, atau satu kasus belum ditangani	Console masih menampilkan galat, atau galat dihilangkan dengan menghapus baris
Kebersihan kode, deklarasi AI, dan commit	15%	Penamaan konsisten, indentasi rapi, tidak ada kode mati; deklarasi AI terisi di README; lebih dari satu commit dengan pesan yang menjelaskan	Kode rapi tetapi deklarasi AI belum lengkap, atau hanya satu commit	Deklarasi AI kosong, atau pekerjaan tidak tersimpan di repositori

Yang dinilai adalah pekerjaan Anda sendiri. Topik yang berbeda antar mahasiswa membuat penilaian tidak bisa dibandingkan antar pekerjaan — karena itu yang diperiksa adalah apakah setiap bagian bekerja sebagaimana dimaksud, bukan apakah ia sama dengan contoh.

Lampiran B — Rujukan cepat dan kalau macet

REFERENSI CEPAT JAVASCRIPT hafalkan yang ini

Yang dipakai	Perannya	Kapan dipakai
const	Menampung nilai yang tidak ditunjuk ulang	
let	Menampung nilai yang akan berubah	
`${ }`	Menyusun teks dari nilai	
=== dan !==	Membandingkan nilai beserta tipenya	
??	Nilai bawaan bila nilai sebelumnya null atau undefined	
?.	Mengakses dengan aman bila yang diakses belum ada	
{ ...objek }	Menyalin objek secara dangkal	
[...array]	Menyalin array	

Yang dipakai	Perannya	Kapan dipakai
map / filter / find	Mengubah / menyaring / mengambil satu isi	
console . table	Melihat seluruh isi array	

KALAU MACET cari gejalanya di sini

Gejala	Penyebab yang paling sering	Yang dilakukan
Halaman tampak normal, tetapi tidak ada satu pun hasil di Console	Berkas app.js kosong, atau baris <script> tidak ada	Periksa isi app.js dan baris <script> sebelum </body>
Console: 404 (Not Found) untuk app.js	Jalur di atribut src tidak cocok dengan letak berkasnya	Pastikan ada folder js/ berisi app.js, dan huruf besar-kecilnya sama
Console: blocked by CORS policy	Halaman dibuka dengan klik dua kali, bukan lewat server lokal	Buka lewat Live Server atau python3 -m http.server 8000
Hasil map berisi undefined semua	Callback map tidak mengembalikan nilai	Tambahkan return, atau tulis arrow function satu baris tanpa kurung kurawal
Perubahan pada salinan ikut mengubah data asli	Objek disalin dengan = , bukan { ... }	Ganti menjadi const salinan = { ...profil }
Nilai dari kolom isian tidak bisa dihitung	Nilai dari input selalu berupa teks	Ubah dengan Number(input.value) sebelum dihitung
Cannot read properties of null	querySelector tidak menemukan elemennya	Bandingkan penulisan id atau class dengan yang ada di HTML
Fungsi memberi hasil undefined	Tidak ada return, atau return-nya berada di dalam forEach	Pindahkan return ke akhir fungsi, atau ganti forEach dengan map

Halaman yang Anda bangun hari ini adalah bahan ujian CPMK Blok 2 nanti: DOM, event, dan pengolahan data semuanya berjalan di atas data yang Anda susun di lembar D. Simpan pekerjaan ini rapi di repositori Anda — Anda akan memakainya lagi.

LAMPIRAN — MENYIMPAN PEKERJAAN ANDA KE GIT sekali saja, bukan tiap pertemuan

Lampiran ini bukan pengantar Git. Panduan Git dan GitHub yang sudah Anda pakai sejak Pertemuan 3 masih berlaku seluruhnya; bagian ini hanya mengulang tiga perintah yang paling sering dipakai, karena mulai hari ini ada berkas baru yang harus ikut disimpan.

Tiga perintah yang akan Anda pakai terus

Perintah	Artinya	Kapan dijalankan
git add .	Menandai seluruh perubahan di folder ini untuk disimpan	Setiap kali selesai satu lembar
git commit -m "pesan"	Menyimpan perubahan itu sebagai satu riwayat	Langsung setelah git add
git push	Mengirim riwayat itu ke GitHub	Setelah commit, supaya dosen bisa melihatnya

Titik pada git add . Titik itu berarti "seluruh berkas di folder ini". Tanpa titik, Git tidak tahu apa yang harus disimpan. Hal yang sama berlaku untuk folder js/ yang baru Anda buat — ia ikut tersimpan karena berada di dalam folder semester.

Berapa kali harus menyimpan?

Tidak ada angka yang benar, tetapi patokannya begini: bila Anda sudah menyelesaikan satu hal yang berarti dan bisa Anda jelaskan, simpanlah. Pada pertemuan ini minimal dua kali: setelah data dipindahkan, dan setelah fungsi serta array methods selesai.

Kalau ini terjadi	Yang Anda lakukan
Sudah menulis banyak, belum pernah commit	Simpan sekarang juga
Salah commit, belum push	Perbaiki lalu commit lagi — jangan hapus repositori
Sudah push, lalu menemukan kekeliruan	Perbaiki berkasnya dan commit lagi; riwayat lama tidak hilang dan itu wajar

Yang dinilai bukan jumlah commit. Menyimpan sering membuat pekerjaan Anda tidak bergantung pada satu berkas di satu komputer — dan pada pertemuan ujian, repositori itulah yang menentukan apakah pekerjaan Anda bisa dinilai.